

機器・ファーム仕様（202609）

アプリが前提としている受信機側の仕様。アプリ（受信側）の仕様は `funcspec-202609.md`、実装構成は `design-202609.md` を参照。ここに書かれた値を変更した場合は、アプリ側（`js/transport.js` / `js/nmea.js`）と必ず対応させること。

1. 構成

```
[u-blox MAX-M10S] --UART(38400 8N1)--> [Raspberry Pi Pico W] --BLE(NUS notify)--> [Android Chrome / 本PWA]
```

要素	役割
MAX-M10S	マルチGNSS（GPS / GLONASS / Galileo / BeiDou / QZSS）受信。NMEA を 1 Hz で出力
Pico W	生 NMEA を一切パースせずそのまま BLE で配信する「ダムパイプ」。ファーム = <code>micropython/main.py</code>
本アプリ	パース・判定・記録・解析のすべてを担う

- ネットワーク不要。ルーターや PC が無い屋外でも Pico W と Android だけで完結する。
- WiFi / WebSocket 経路は廃止済み（`config.py` も不要）。BLE 専用。
- iPhone / iPad は Web Bluetooth 非対応のため接続できない。

配線

Pico W	向き	MAX-M10S
GP0（UART0 TX）	→	RX
GP1（UART0 RX）	←	TX

2. UART（Pico W ↔ MAX-M10S）

項目	値
目標ボーレート	38400
工場出荷時ボーレート	9600（M10S は設定を保存しないため電源投入時はこちら）
受信バッファ	4096 バイト（BLE 送信のブロック中に溢れないよう拡大）

38400 にする理由: 9600 ではマルチGNSSのフル出力（特に GSV）が帯域に収まらず間引かれ、スカイプロット・SNR チャートに全系統の衛星が出そろわない。

ボーレートの自動同期（起動時・最大 3 回試行）

1. 38400 で開き、1.5 秒以内にチェックサムの正しい NMEA 行が来るか確認する（来れば確定）。
2. 来なければ 9600 で開いて確認し、受けられたら **UBX-CFG-VALSET**（キー **CFG-UART1-BAUDRATE = 0x40520001**、layers = RAM|BBR）で 38400 への切替を指示し、200 ms 待つ。
3. 次の試行の先頭で 38400 を再確認する（＝切替の検証）。
4. 3 回で確定できなければ 9600 のまま継続する（GNSS 未接続・故障時も動作を止めない）。

3. BLE（Pico W ↔ Android）

Nordic UART Service（NUS）＝標準的な「シリアル over BLE」。アプリ側と一致させること。

項目	値
広告名（デバイス名）	picow （アプリは名前の完全一致でフィルタする）
Service UUID	6E400001-B5A3-F393-E0A9-E50E24DCCA9E
TX（周辺 → 中央 / notify 。受信に使う）	6E400003-B5A3-F393-E0A9-E50E24DCCA9E
RX（中央 → 周辺 / write）	6E400002-B5A3-F393-E0A9-E50E24DCCA9E （未使用。NUS の形を保つため登録のみ）
広告間隔	200 ms

送信の単位と分割

- 1 行（NMEA 1 センテンス）の末尾に **\n** を付けて送る。
- notify 1 回のペイロードは **ATT_MTU - 3**（既定 20 バイト）。1 行はこれに収まるよう分割される。MTU 交換が行われた場合はその値に追従する。
- したがって受信側では「チャンクを連結 → **\n**（および ***XX**）で行を復元」する必要がある（アプリ側は **LineBuffer** が担当）。
- 送信バッファが詰まった場合（**OSError**）は 5 ms 間隔で最大 10 回リトライする。それでも送れない行は諦めて次へ進む（接続は切らない）。途切れた行は受信側のチェックサム検証で弾かれる。
- 切断されると自動で再広告し、次の端末を受け付ける。

ダムパイプの原則

- チェックサム NG の行もそのまま転送する（判定は受信側で行い、件数を数えて破棄する）。
- ファームは NMEA の中身を解釈しない。したがって「欲しい情報が増えてもファーム変更は不要」。

4. 受信品質カウンタ \$PPICO

ダムパイプの唯一の例外。Pico 側の通過数・破棄数を 5 秒ごと（**接続中のみ**）に配信する。アプリはこれを統計にのみ用い、エポック組み立てには渡さない。

```
$PPICO,<seq>,<rx>,<ng>,<drop>,<txok>,<txng>*<CS>
```

フィールド	意味
seq	統計文の通し番号（1 から）。抜けを見れば統計文自体の欠落が分かる
rx	UART から受信した行数（起動からの累計）
ng	うちチェックサム NG（M10S → Pico 間の欠落・化けの兆候）
drop	行にならないままバッファ（2048 バイト超）を破棄した回数
txok	BLE へ送信完了した行数（\$PPICO 自身を含む）
txng	リトライしても送れず破棄した行数

- カウンタは **snapshot** → **送信** の順に処理するため、その文の **txok** は自身を含まない。
- チェックサムは本体（\$ と * の間）の XOR、16 進 2 桁。アプリ側の検証と同一。
- 利用方法（BLE 欠落の推定式）は [algospec-202609.md](#) 5. を参照。

5. 堅牢化（屋外無人運用向け）

仕組み	内容
ウォッチ ドッグ	8 秒（RP2040 の上限付近）。主ループで毎回リセットする。初期化完了後に起動するため、UART 自動同期の待ち時間で誤リセットしない
メモリ回 収	2 秒ごとに <code>gc.collect()</code>
BLE 再広告	切断検知時に自動で再開
送信リト ライ	3. のとおり。1 行を諦めても接続は維持する
主ループ	10 ms スリープを挟む

LED での状態表示

LED	状態
点灯	正常動作中（BLE 広告開始後）
0.5 秒周期の点滅（継続）	bluetooth モジュールが無いファーム（BLE 非対応）。動作不可
2 回速く点滅 → 0.7 秒休み（継続）	BLE 初期化に失敗

6. アプリ側との対応表

ファームを変更する場合、次の箇所を必ず合わせること。

ファーム（ <code>micropython/main.py</code> ）	アプリ
<code>BLE_NAME = "picow"</code>	<code>js/transport.js</code> の <code>DEVICE_NAME</code>
<code>_NUS_UUID_STR / _NUS_TX_STR</code>	<code>js/transport.js</code> の <code>NUS_SERVICE / NUS_TX</code>

ファーム (micropython/main.py)	アプリ
行末 \n + ATT_MTU 分割	js/nmea.js の LineBuffer (\n と *XX で行復元)
チェックサム (\$~* の XOR)	js/nmea.js の xorChecksum / validateChecksum
\$PPICO のフィールド順	js/nmea.js の parsePPICO、js/stream-stats.js
NMEA の種類 (無加工で全種)	js/nmea.js の PARSED_TYPES (GGA/RMC/GSA/GSV/VTG/GST)

7. 運用上の注意

- 電源投入直後は M10S が 9600 で起動するため、ボーレート同期に数秒かかる。
- 初回測位（コールドスタート）は開けた空でも数十秒～数分かかる。
- レシーバは静止させ、アンテナを上に向けて空を遮らない位置に置く。
- 測定中はスマホの画面を点けたままにする（画面 OFF で BLE が切断される）。